

NERVI

Project Report

Local LLM Inference Platform for Intel NPU, Built from Scratch

Author: Durga Sai (dsainvg)

GitHub: github.com/dsainvg

HuggingFace: [durgasai299792458](https://huggingface.co/durgasai299792458)

Personal Site: dsainvg.me

Project Links

Resource	Link
Project overview	https://projects.dsainvg.me/nervi
NERVI CLI repo	https://github.com/dsainvg/NERVI
NERVI docs and marketplace	https://nervi.dsainvg.me
npuserver repo	https://github.com/dsainvg/npuserver
npuserver docs	https://dev.dsainvg.me/npuserver/
Training data repo	https://github.com/dsainvg/data_qwen3.5_4B_finetune
HuggingFace models	https://huggingface.co/durgasai299792458
Extension marketplace	https://nervi.dsainvg.me/marketplace
Fine-tune and conversion scripts	https://github.com/dsainvg/nervi-finetunes-and-convertors
OpenVINO model conv (abandoned)	https://github.com/dsainvg/openvino-model-conv

Overview

NERVI is a fully self-built local AI inference platform that enables running large language models entirely offline on Intel NPU hardware. Every mainstream local LLM tool available today targets GPU inference on CUDA or Metal. The Intel AI Boost NPU present in Core Ultra laptops sits completely idle because no tooling exists for it. NERVI is built to close that gap, covering everything from raw model training data to a live user-facing CLI and extension marketplace.

The platform was built entirely by a single developer and spans model fine-tuning, NPU conversion, a Python inference server, a Go CLI, an extension system, and a hosted marketplace. Every layer required original engineering rather than configuration of existing tools.

1. Model Pipeline

Running models on Intel NPU is not a matter of pointing existing tools at a HuggingFace checkpoint. NPU inference via OpenVINO has hard constraints that no open-source workflow handles by default, so the entire model pipeline was built from scratch.

1.1 Synthetic Dataset

A custom training dataset was created specifically for NERVI's interaction protocol. The dataset consists of synthetic multi-turn conversation traces that teach models to respond correctly to NERVI's tool-call and menu-based interaction format. Generic instruction-following datasets do not produce the structured, parseable responses that NERVI's extension IPC requires. The dataset is published at github.com/dsainvg/data_qwen3.5_4B_finetune.

1.2 Fine-Tuning

Models were fine-tuned on Kaggle free-tier GPUs using QLoRA via HuggingFace Transformers and PEFT. The primary base model is Qwen3 4B, selected for the balance between capability and size that fits within NPU SRAM at INT4 quantization. Additional base models were also fine-tuned to provide variety in the NERVI model registry. The QLoRA adapter is merged before conversion so the final model is a single self-contained checkpoint.

1.3 NPU Conversion

Converted models are exported to OpenVINO IR format using optimum-cli from the optimum-intel library. The critical constraint discovered during this work is that Intel NPU blob compilation requires symmetric INT4 quantization. The default quantization mode used by virtually every GPU and CPU pipeline is asymmetric, which passes validation silently but produces a compiled blob that generates incorrect outputs. All NERVI model conversions enforce symmetric INT4 with a ratio of 1.0 and group size of 128.

Converted models are published as complete HuggingFace repositories under durgasai299792458, containing the OpenVINO IR weights, tokenizer IR files, and configuration. Five-plus models have been fine-tuned, converted, and published.

2. npuserver

npuserver is a Python library and Flask REST server that manages the full model lifecycle on Intel NPU hardware. It is distributed as a PyPI package and serves as the inference backend that NERVI's CLI manages as a subprocess.

2.1 Model Lifecycle

Intel NPU SRAM is a constrained resource that cannot be overcommitted or paged. npuserver models this explicitly with a four-state lifecycle: available (known remotely, not yet downloaded), downloaded (weights cached locally), compiled (NPU blob files ready for instant load), and active (currently occupying NPU memory). Only one model can be active at a time, and unloading triggers explicit memory cleanup to prevent SRAM leaks.

When a downloaded but not-yet-compiled model is loaded, npuserver intercepts the request, runs OpenVINO GenAI's LLMPipeline targeting the NPU device to produce device-specific blob files, writes a sentinel on success, and proceeds to load. Subsequent loads skip compilation entirely and are near-instant. Compilation is a one-time cost per model per device.

2.2 Inference and API

npuserver exposes an OpenAI-compatible REST API, implementing the /v1/chat/completions endpoint with both blocking and SSE streaming response modes. The streaming path pipes OpenVINO GenAI token callbacks directly to the HTTP response stream. The API response includes a metrics object with time-to-first-token and throughput figures from the NPU.

Because the API surface matches the OpenAI schema, any existing tool that accepts a custom base URL connects to NERVI's NPU inference without code changes. This was a deliberate forward-looking decision to make npuserver useful beyond the NERVI CLI.

Inference throughput: 13.1 tok/s on DeepSeek-R1-Distill 1.5B, 10.1 tok/s on Llama 3.2 3B, 6.4 tok/s on Qwen3 4B. Model load time is 5 to 10 seconds depending on model size.

3. NERVI CLI

The CLI is distributed as a single compiled Go binary. On first run it bootstraps a managed Python virtual environment, installs `npuserver` from PyPI, and writes a sentinel file so subsequent runs skip setup entirely. Users never need touch Python or pip directly.

The interactive chat shell is implemented with raw ANSI escape codes rather than a third-party TUI library. Viewport isolation is achieved by restricting the terminal scroll region to all rows except the bottom status bar, preventing the bar from duplicating on scroll events. For reasoning models that emit think tokens before their response, the shell shows a spinner during the thinking phase and replaces it with an elapsed time readout when the first response token arrives. Window size detection is implemented separately for Windows (Console API) and Unix (ioctl).

The CLI covers the full lifecycle: pulling and compiling models, loading and unloading from NPU memory, checking hardware telemetry, managing configuration and model nicknames, and the complete extension management surface including install, scaffold, enable, disable, log tailing, and real-time state monitoring.

```
Windows PowerShell | x - + -
```

```
> dirpath\209792438\qemu-48-cvnnas-lora available
> dirpath\209792438\qemu-48-OpenTNO-INTU-rpu available
> dirpath\209792438\qemu2.5-48-OpenTNO-INTU-rpu available
> dirpath\209792438\qemu2.5-48-OpenTNO-INTU-rpu available
> dirpath\209792438\qemu2.5-48-int-4-openline-rpu available
> dirpath\209792438\qemu2.5-48-OpenTNO-NFO-MODEL available
> dirpath\209792438\qemu2.5-48-qnet-FineTune available
> dirpath\209792438\qemu2.5-48-OpenTNO-INTU available

~/Load qemu6

> Model dirpath\209792438\qemu2.5-48-OpenTNO-INTU-rpu.rpi compiled and loaded successfully!

> Hello how are you doing and say me about Neural Processing unit in Intel processors and what's your take on it?

+ Thought: G.W.
Hello! I'm doing great, thank you for asking - I'm always here to chat and help out! 🌟

Now, about **Neural Processing Units (NPU)** in Intel processors:
```

What is a Neural Processing Unit (NPU)?

A Neural Processing Unit (NPU) is a specialized hardware component designed to accelerate machine learning workloads - especially those involving artificial intelligence (AI), deep learning, and neural network computations. Unlike traditional CPUs, which are optimized for general-purpose tasks, NPUs are tailored for the parallel, matrix-based operations common in AI inference and training.

Does Intel Have an NPU?

Yes - Intel does have a neural processing unit, though it's not as prominently marketed as some other companies (like Apple with its Neural Engine or Google with Tensor Processors).

Intel's NPU is primarily found in **Intel's Advanced Matrix Extensions (AMX)**, introduced in **Intel 4th Gen (Sapphire Rapids)** and later generations (like **Intel Core i7-13700H** and beyond).

Key Points:

- The **AMX (Advanced Matrix Extensions)** is Intel's implementation of a specialized neural processing unit.
- AMX is designed to accelerate **matrix operations** - the core of deep learning - using **8x8, 16x16, and 32x32 vector operations**.
- It's optimized for **AI inference**, computer vision, natural language processing (NLP), and edge AI applications.
- AMX is **not** a full-fledged GPU-like in mobile chips (e.g., Apple M-series), but it's a **hardware acceleration unit** that offloads AI workloads from the CPU.

How Is NPU Used?

- Enabling AI models on your PC (e.g., image classification, voice assistants).
- Real-time video analysis (e.g., face detection, object tracking).
- Large language model (LLM) inference (e.g., running local LLMs like Llama, Mistral, or even GPT-3.5 via local models).
- Enhanced performance in AI-powered apps: Adobe Photoshop, Microsoft Office with AI features, etc.

My Take (as an AI assistant):

- Praise of Intel's NPU (AMX):
- A significant step toward **AI-native computing** in desktops and laptops.
- Great for **power-efficient AI** workloads -

```
--- CPU @ 3.8 GHz --- Total 31.9s --- Latency 87B --- T.B. 140/s ---
```

```
@ NPU2 qemu2-48 - idle
```

4. Extension System

NERVI supports a plugin architecture that lets developers extend the CLI without modifying the core binary. Extensions run as isolated OS subprocesses and can be written in any language that produces an executable.

4.1 IPC Protocol

The CLI communicates with extensions over newline-delimited JSON on stdin and stdout. Every message carries an ID field, which allows the stateful broker to correctly multiplex concurrent requests to long-running background extensions. Crashes in an extension process do not affect the CLI. Standard error is captured to per-extension log files accessible via `nervi app logs`.

4.2 Interaction Modes

Extensions declare one of two interaction modes. Tool mode exposes callable functions that the CLI or an agentic model can invoke with typed arguments and receive structured results. Menu mode provides a text-driven interactive interface where the extension returns a title, a list of selectable items, and optional text content, and the CLI handles the interaction loop. Stateless extensions spawn per request and exit immediately. Stateful extensions run as background daemons and handle multiple concurrent calls.

4.3 Developer Experience

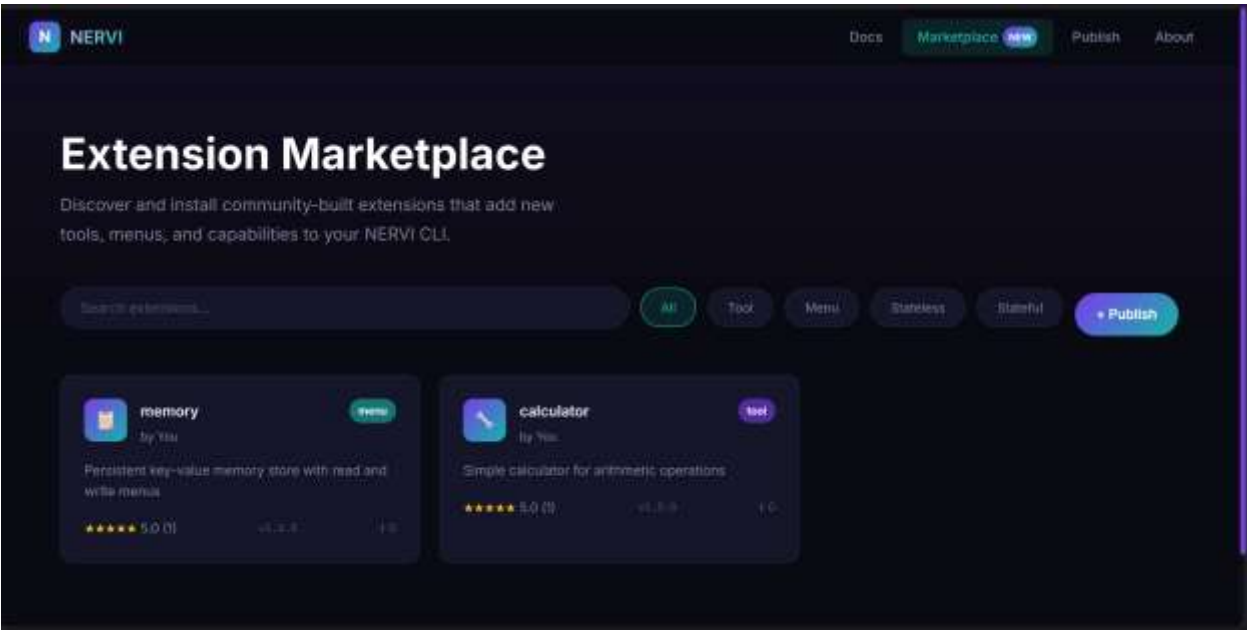
`nervi app create` scaffolds a boilerplate extension with a manifest file and a Go entry point already wired to the JSON framing protocol. The manifest declares the extension name, version, mode, lifecycle type, and tool schemas. `nervi app monitor` provides real-time observation of a stateful extension's persisted state.

5. Extension Marketplace

A live extension marketplace at nervi.dsainvg.me/marketplace provides discovery and install for NERVI extensions. The backend uses Cloudflare Workers for the API layer and Appwrite for extension file storage and registry data. The frontend is statically hosted.

Developers publish extensions by packaging the compiled binary and manifest as a zip and uploading via the marketplace site. Users install with `nervi app install`, which fetches and extracts the zip and registers the extension by reading the manifest.

Two reference extensions ship with the platform. The memory extension demonstrates stateful menu-mode interaction, providing a persistent key-value store with read and write menus. The calculator extension demonstrates stateless tool-mode interaction, exposing an arithmetic function callable by the CLI or an agent.



6. Architecture Overview

Component	Technology	Role
Synthetic dataset	Custom JSONL	Training corpus for NERVI-specific agentic behaviour
Fine-tuning	HF Transformers + PEFT QLoRA, Kaggle GPUs	Qwen3 4B and other base models adapted to NERVI's protocol
NPU conversion	optimum-cli, OpenVINO IR	Symmetric INT4 export and HuggingFace publishing
npuserver	Flask + OpenVINO GenAI	Model lifecycle, blob compilation, OpenAI-compatible API
Intel AI Boost NPU	Core Ultra (Meteor Lake+)	On-device LLM execution without GPU
NERVI CLI	Go	TUI shell, server management, extension broker
Extension marketplace	Cloudflare Workers + Appwrite	Serverless API, extension file storage and registry

7. Engineering Highlights

Symmetric INT4 quantization requirement

Asymmetric INT4, the default for virtually every open-source quantization tool, silently produces incorrect NPU blob compilation outputs. There is no error, no warning, and the blob appears valid. Identifying this and enforcing symmetric quantization throughout the conversion pipeline was the key technical discovery that made NPU inference work at all.

End-to-end model ownership

No suitable NPU-ready model existed on HuggingFace. The project required building the entire supply chain: writing the training data, fine-tuning with QLoRA on Kaggle GPUs, converting to symmetric INT4 OpenVINO IR, and publishing. Five-plus models have been produced and published publicly.

Explicit NPU memory lifecycle

GPU inference servers can lazily swap models because GPU memory is virtualised and models can be partially evicted. Intel NPU SRAM has neither property. npuserver's four-state lifecycle (available, downloaded, compiled, active) and explicit load/unload contract prevent the OOM failures that naive implementations hit silently.

Go and Python split with zero user-facing setup

Distributing a single Go binary while retaining full access to OpenVINO's Python-only bindings required the two-process architecture. The auto-bootstrap mechanism on first run hides all Python dependency management from the user entirely.

Language-agnostic extension protocol

Using JSON over stdio rather than HTTP or gRPC means extension authors have no framework dependency. Any executable that reads from stdin and writes to stdout is a valid NERVI extension. The ID field on every message enables the stateful broker to multiplex concurrent requests correctly without a separate connection layer.

8. Dead Ends and Abandoned Approaches

NPU conversion of a Mamba-architecture model (Qwen3.5 4B)

An earlier attempt was made to extend NERVI's model pipeline to Qwen3.5 4B, which includes Mamba-style state-space layers alongside the standard transformer blocks. The work is tracked at github.com/dsainvg/opencvino-model-conv.

The blocker was immediate: the Intel NPU driver does not support the Mamba activation operations used by these layers. OpenVINO's model compilation rejects them at blob generation time with no fallback path. A hybrid execution strategy was then attempted keeping the Mamba layers on the CPU while routing the remaining transformer blocks to the NPU. This is architecturally possible with OpenVINO's device assignment API, and the model ran without errors.

However, the per-token data transfer overhead between CPU and NPU at the layer boundaries destroyed throughput completely. Inference became slower than running the model entirely on CPU, making the hybrid approach worse in every metric than the baseline. The approach was abandoned. Pure transformer architectures which is what all currently published NERVI models use do not hit this constraint and compile and run on NPU cleanly.

9. Outcomes

- **Inference performance:** 13.1 tok/s on DeepSeek-R1-Distill 1.5B, 10.1 tok/s on Llama 3.2 3B, 6.4 tok/s on Qwen3 4B, all running fully offline on Intel NPU without any GPU.
- **Model pipeline:** Five-plus models fine-tuned with QLoRA on Kaggle GPUs, converted to symmetric INT4 OpenVINO IR, and published publicly on HuggingFace under [durgasai299792458](https://huggingface.co/durgasai299792458).
- **Compatibility:** OpenAI-compatible API surface on npuserver means any LLM tool (LangChain, AutoGen, openai-python) connects to NERVI's NPU inference with a one-line base URL change.
- **Ecosystem:** Live extension marketplace with publish, install, scaffold, enable, disable, log, and monitor tooling, shipping reference extensions that demonstrate both interaction modes.
- **Documentation:** Complete public documentation at nervi.dsainvg.me covering all components, with a full REST API reference for npuserver.

All components are open-source and publicly accessible. Source code, documentation, models, and marketplace are live.